Exercise 2: Semantic Search System Using Cosine Similarity

Aim

To develop a semantic search system that ranks documents by the cosine similarity between document embeddings and a query embedding.

Algorithm

1. Import the required libraries (sentence-transformers, numpy).
2. Load a pre-trained embedding model (all-MiniLM-L6-v2).
3. Define the document collection and convert each document into an embedding vector.
4. Accept a user query and convert it into an embedding vector.
5. Compute the cosine similarity between the query vector and every document vector.
6. Sort the documents by similarity score in descending order.
7. Display the top-k results with their scores.

Python Code

```python
from sentence_transformers import SentenceTransformer
import numpy as np

class SemanticSearch:
    def __init__(self, documents, model_name="all-MiniLM-L6-v2"):
        self.documents = documents
        self.model = SentenceTransformer(model_name)
        # Embed all documents once
        self.doc_embeddings = self.model.encode(documents)

    @staticmethod
    def cosine_similarity(a, b):
        return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

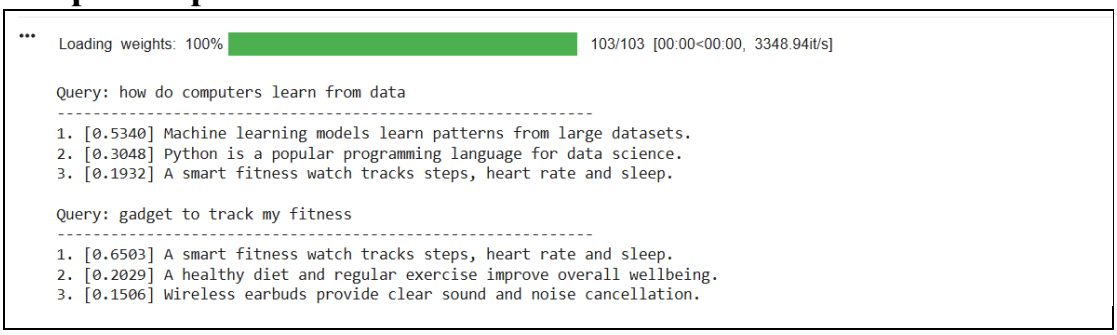
    def search(self, query, top_k=3):
        query_embedding = self.model.encode(query)
        scores = [self.cosine_similarity(query_embedding, d)
                  for d in self.doc_embeddings]
        ranked = np.argsort(scores)[::-1][:top_k]
        return [(self.documents[i], float(scores[i])) for i in ranked]


# Sample document collection
documents = [
    "Python is a popular programming language for data science.",
    "Machine learning models learn patterns from large datasets.",
    "A smart fitness watch tracks steps, heart rate and sleep.",
    "Wireless earbuds provide clear sound and noise cancellation.",
    "A healthy diet and regular exercise improve overall wellbeing.",
    "The stock market closed higher after strong earnings reports.",
]


engine = SemanticSearch(documents)

for query in ["how do computers learn from data",
              "gadget to track my fitness"]:
    print(f"\nQuery: {query}")
    print("-" * 60)
    for rank, (doc, score) in enumerate(engine.search(query), start=1):
        print(f"{rank}. [{score:.4f}] {doc}")
```

Sample Output



Result

The semantic search system ranked documents by cosine similarity between query and document embeddings, returning the most relevant matches by meaning rather than exact keywords.